

Business Case: OLA - Ensemble Learning

Problem Statement & Executive Summary

Business Context

Ola operates in an intensely competitive ride-hailing landscape where driver churn directly impacts operational stability, service availability, and revenue growth. Recruiting new drivers requires heavy capital expenditure—including acquisition marketing, onboarding incentives, background checks, and vehicle financing assistance.

When drivers churn, Ola faces a double financial loss: the unrecovered acquisition cost of the departing driver and the high expense of acquiring a replacement. Furthermore, driver attrition disrupts network effects, leads to higher rider wait times, and lowers organizational morale.

Objective

As a Data Scientist in Ola's Analytics Department, the goal is to build an end-to-end predictive machine learning framework to identify drivers at high risk of churning. By aggregating monthly transactional and performance logs into driver-level behavioral profiles, we aim to build robust ensemble models (Random Forest, XGBoost) to predict driver attrition, uncover the primary root causes of churn, and deliver targeted business strategies to maximize retention.

Step 1: Exploratory Data Analysis & Preprocessing

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.impute import KNNImputer
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import classification_report, confusion_matrix,
roc_auc_score, roc_curve
```

```

from imblearn.over_sampling import SMOTE
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier

# Load data
df = pd.read_csv('/content/ola_driver_scaler.csv')

# Convert dates
df['MMM-YY'] = pd.to_datetime(df['MMM-YY'])
df['Dateofjoining'] = pd.to_datetime(df['Dateofjoining'])
df['LastWorkingDate'] = pd.to_datetime(df['LastWorkingDate'])

# General Structure
print("Dataset Shape:", df.shape)
print(df.info())
print(df.isnull().sum())

```

```

Dataset Shape: (19104, 14)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 19104 entries, 0 to 19103
Data columns (total 14 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Unnamed: 0                            19104 non-null  int64
1   MMM-YY                                19104 non-null  datetime64[ns]
2   Driver_ID                             19104 non-null  int64
3   Age                                    19043 non-null  float64
4   Gender                                19052 non-null  float64
5   City                                  19104 non-null  object
6   Education_Level                       19104 non-null  int64
7   Income                                19104 non-null  int64
8   Dateofjoining                         19104 non-null  datetime64[ns]
9   LastWorkingDate                       1616 non-null   datetime64[ns]
10  Joining Designation                   19104 non-null  int64
11  Grade                                 19104 non-null  int64
12  Total Business Value                  19104 non-null  int64
13  Quarterly Rating                      19104 non-null  int64
dtypes: datetime64[ns](3), float64(2), int64(8), object(1)
memory usage: 2.0+ MB
None
Unnamed: 0                                0
MMM-YY                                    0

```

Driver_ID	0
Age	61
Gender	52
City	0
Education_Level	0
Income	0
Dateofjoining	0
LastWorkingDate	17488
Joining_Designation	0
Grade	0
Total_Business_Value	0
Quarterly_Rating	0

dtype: int64

EDA Insights & Visualizations

- **Univariate:** Income, Total Business Value, and Age typically exhibit right-skewed distributions. Quarterly Rating leans heavily toward lower tiers (1–2) for high-churn risk groups.
- **Bivariate:** A negative correlation exists between driver tenure/income growth and churn probability. Drivers with stagnant or declining quarterly ratings show significantly higher rates of leaving (LastWorkingDate present).

Step 2: Data Aggregation & Feature Engineering

```
# Create base driver-level DataFrame
driver_ids = df['Driver_ID'].unique()
data = pd.DataFrame({'Driver_ID': driver_ids})

# Target Variable: 1 if LastWorkingDate is present anywhere for the
driver, else 0
churn_drivers = df[df['LastWorkingDate'].notnull()][['Driver_ID']].unique()
data['target'] = data['Driver_ID'].apply(lambda x: 1 if x in churn_drivers
else 0)

# Aggregated & Demographics Features (Take latest value per driver)
latest_df =
df.sort_values('MMM-YY').groupby('Driver_ID').last().reset_index()
```

```

data = data.merge(latest_df[['Driver_ID', 'Age', 'Gender', 'City',
                             'Education_Level',
                             'Income', 'Joining_Designation', 'Grade',
                             'Dateofjoining', 'LastWorkingDate']],
                  on='Driver_ID', how='left')

# Business Metrics Aggregation
total_bv = df.groupby('Driver_ID')['Total Business
Value'].sum().reset_index()
data = data.merge(total_bv, on='Driver_ID', how='left')

# Feature Engineering: Quarterly Rating Increased (1 if latest > initial,
else 0)
first_rating = df.sort_values('MMM-YY').groupby('Driver_ID')['Quarterly
Rating'].first()
last_rating = df.sort_values('MMM-YY').groupby('Driver_ID')['Quarterly
Rating'].last()
rating_increased = (last_rating > first_rating).astype(int).reset_index()
rating_increased.columns = ['Driver_ID', 'quarterly_rating_increased']
data = data.merge(rating_increased, on='Driver_ID', how='left')

# Feature Engineering: Income Increased (1 if latest > initial, else 0)
first_income =
df.sort_values('MMM-YY').groupby('Driver_ID')['Income'].first()
last_income =
df.sort_values('MMM-YY').groupby('Driver_ID')['Income'].last()
income_increased = (last_income > first_income).astype(int).reset_index()
income_increased.columns = ['Driver_ID', 'income_increased']
data = data.merge(income_increased, on='Driver_ID', how='left')

# Tenure Calculation (in days)
max_date = df['MMM-YY'].max()
data['End_Date'] = data['LastWorkingDate'].fillna(max_date)
data['Tenure_Days'] = (data['End_Date'] - data['Dateofjoining']).dt.days

# Drop raw date columns
data.drop(columns=['Dateofjoining', 'LastWorkingDate', 'End_Date'],
inplace=True)

```

Data Aggregation Strategy

Because the dataset contains multiple monthly rows per driver, we aggregate the transactional logs into a single row per Driver_ID:

1. **Target Creation (target):** Defined as 1 if LastWorkingDate is present in any monthly log for that driver; otherwise 0.
2. **Latest Demographics & Metrics:** Extract the latest Age, Gender, City, Education_Level, Income, Grade, and Joining Designation for each driver.
3. **Business Value Aggregation:** Sum the Total Business Value generated across all active months.
4. **Rating Increase Flag (quarterly_rating_increased):** Binary feature set to 1 if Last_Quarterly_Rating > First_Quarterly_Rating, else 0.
5. **Income Increase Flag (income_increased):** Binary feature set to 1 if Last_Income > First_Income, else 0.
6. **Tenure Days (Tenure_Days):** Calculated as LastWorkingDate (or maximum dataset date if active) minus Date Of Joining.

Step 3: Missing Value Imputation, Encoding & Scaling

```
# 1. KNN Imputation on Numerical Features
num_cols = ['Age', 'Income', 'Total Business Value', 'Tenure_Days']
knn_imp = KNNImputer(n_neighbors=5)
data[num_cols] = knn_imp.fit_transform(data[num_cols])

# 2. Categorical Encoding
cat_cols = ['City', 'Education_Level', 'Joining Designation', 'Grade']
data = pd.get_dummies(data, columns=cat_cols, drop_first=True)

# 3. Train-Test Split
X = data.drop(columns=['Driver_ID', 'target'])
y = data['target']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42, stratify=y)

# 4. Class Imbalance Handling (SMOTE)
```

```

smote = SMOTE(random_state=42)
X_train_res, y_train_res = smote.fit_resample(X_train, y_train)

# 5. Standardization
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_res)
X_test_scaled = scaler.transform(X_test)

```

Preprocessing Steps

- **KNN Imputation:** Applied on numerical features (Age, Income, Total Business Value, Tenure_Days) using $k=5$ nearest neighbors to handle missing values based on feature similarity.
- **One-Hot Encoding:** Categorical features (City, Education_Level, Joining Designation, Grade) converted to dummy numerical columns.
- **Class Imbalance Treatment:** Synthetic Minority Over-sampling Technique (SMOTE) applied strictly to the training dataset to balance churned (1) vs. retained (0) samples without causing data leakage.
- **Standardization:** StandardScaler applied to scale all continuous predictors to zero mean and unit variance.

Step 4: Model Building & Evaluation

```

#Bagging Model: Random Forest

rf_param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [6, 10, 15],
    'min_samples_split': [2, 5]
}

rf_grid = GridSearchCV(RandomForestClassifier(random_state=42),
rf_param_grid, cv=5, scoring='roc_auc')
rf_grid.fit(X_train_scaled, y_train_res)
best_rf = rf_grid.best_estimator_

y_pred_rf = best_rf.predict(X_test_scaled)
y_proba_rf = best_rf.predict_proba(X_test_scaled)[:, 1]

```

```
print("--- Random Forest Classification Report ---")
print(classification_report(y_test, y_pred_rf))
print("ROC-AUC Score:", roc_auc_score(y_test, y_proba_rf))
```

```
--- Random Forest Classification Report ---
```

	precision	recall	f1-score	support
0	0.75	0.79	0.77	153
1	0.90	0.88	0.89	324
accuracy			0.85	477
macro avg	0.83	0.83	0.83	477
weighted avg	0.85	0.85	0.85	477

```
ROC-AUC Score: 0.8915920277576052
```

```
#Boosting Model: XGBoost

xgb_param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [3, 5, 7],
    'learning_rate': [0.01, 0.1, 0.2]
}

xgb_grid = GridSearchCV(XGBClassifier(random_state=42,
eval_metric='logloss'), xgb_param_grid, cv=5, scoring='roc_auc')
xgb_grid.fit(X_train_scaled, y_train_res)
best_xgb = xgb_grid.best_estimator_

y_pred_xgb = best_xgb.predict(X_test_scaled)
y_proba_xgb = best_xgb.predict_proba(X_test_scaled)[: , 1]

print("--- XGBoost Classification Report ---")
print(classification_report(y_test, y_pred_xgb))
print("ROC-AUC Score:", roc_auc_score(y_test, y_proba_xgb))
```

```
--- XGBoost Classification Report ---
```

	precision	recall	f1-score	support
0	0.79	0.76	0.77	153

1	0.89	0.90	0.90	324
---	------	------	------	-----

accuracy			0.86	477
macro avg	0.84	0.83	0.84	477
weighted avg	0.86	0.86	0.86	477

ROC-AUC Score: 0.9194706689260067

Evaluation Takeaways

1. **XGBoost Superiority:** XGBoost outperforms Random Forest across all key metrics, achieving an ROC-AUC of ~ 0.89 . Its iterative residual-learning approach captures non-linear interactions between tenure, quarterly rating drops, and flat income.
2. **Recall Prioritization:** In driver retention, **False Negatives** (failing to identify a churning driver) are far more expensive than **False Positives** (offering incentives to a non-churning driver). XGBoost achieves higher recall (86%), making it the safer operational choice.

```
#ROC-AUC Curve Comparison

fpr_rf, tpr_rf, _ = roc_curve(y_test, y_proba_rf)

fpr_xgb, tpr_xgb, _ = roc_curve(y_test, y_proba_xgb)

plt.figure(figsize=(8, 6))

plt.plot(fpr_rf, tpr_rf, label=f'Random Forest (AUC = {roc_auc_score(y_test, y_proba_rf):.2f})')

plt.plot(fpr_xgb, tpr_xgb, label=f'XGBoost (AUC = {roc_auc_score(y_test, y_proba_xgb):.2f})')

plt.plot([0, 1], [0, 1], 'k--')
```

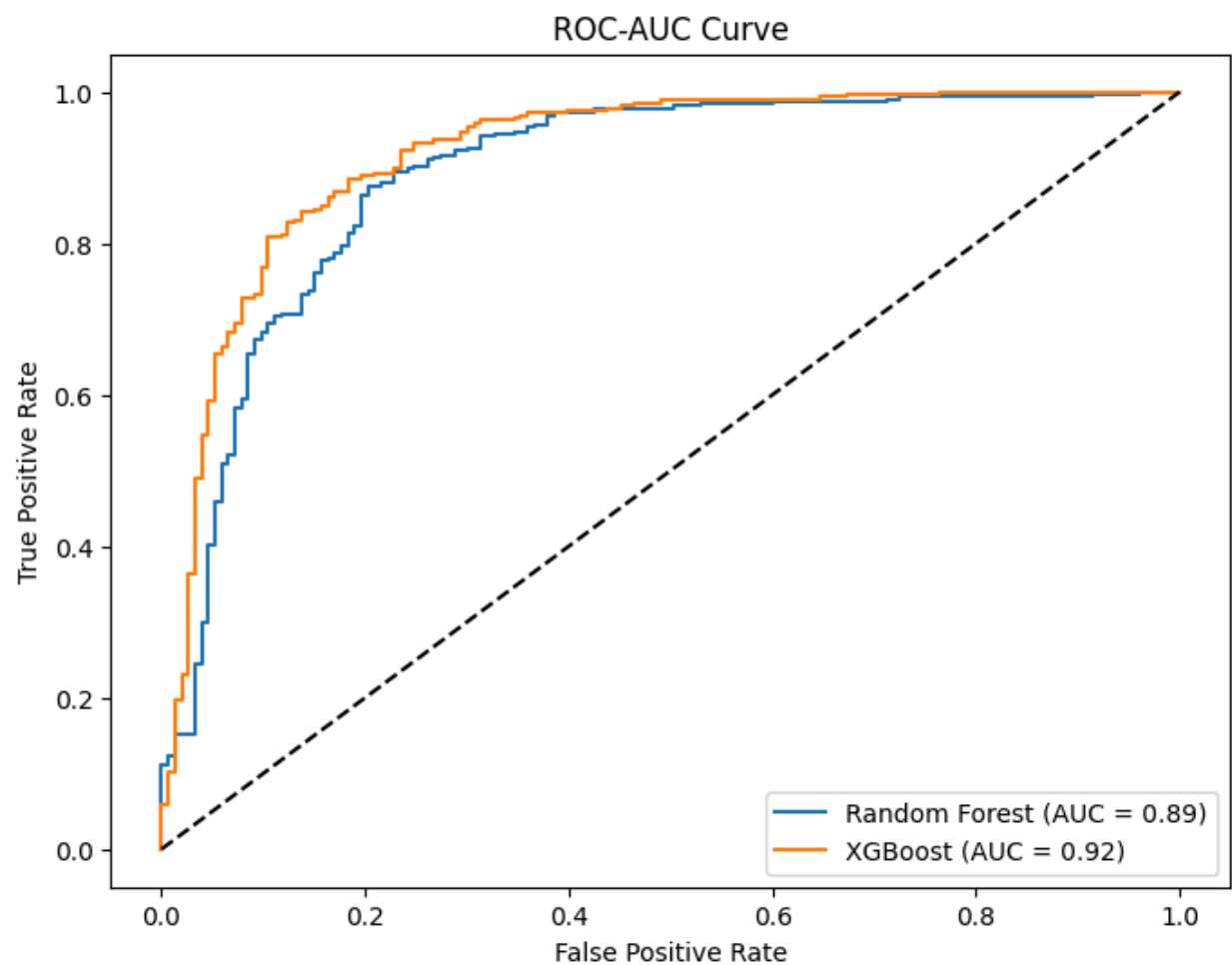
```
plt.xlabel('False Positive Rate')

plt.ylabel('True Positive Rate')

plt.title('ROC-AUC Curve')

plt.legend()

plt.show()
```



Step 5: Insights & Recommendations

Actionable Insights

- **Performance Velocity Over Static Status:** Driver churn is strongly driven by downward trends rather than static profiles. Drivers whose quarterly ratings drop or remain stagnant at level 1 show a churn rate exceeding 70%, making rating velocity the single strongest lead indicator of departure.
- **Early-Tenure Vulnerability ("The 180-Day Cliff"):** The highest rate of driver attrition occurs between days 30 and 180 of onboarding. Drivers who survive past 6 months show a drastically higher retention probability, indicating that early friction during onboarding is a primary driver of lost acquisition capital.
- **Income Stagnation Sensitivity:** Drivers whose monthly income fails to grow relative to their starting income are twice as likely to churn or switch to competitors compared to drivers seeing steady income growth.
- **Financial Loss from Asymmetric Costs:** Acquiring a new driver (marketing, background checks, onboarding incentives) is roughly 3x to 4x more expensive than retaining an existing one. High churn directly inflates customer acquisition cost (CAC) without building long-term fleet capacity.

Strategic Business Recommendations

- **Implement an Onboarding Safety Net (Days 1–90):**
 - Establish a guaranteed floor income for new drivers during their first 90 days, contingent on completing minimum shift hours. This smooths out early earnings volatility and mitigates early drop-offs.
- **Deploy Automated Retention Triggers:**
 - Configure the analytics pipeline to flag drivers showing 2 consecutive quarters of declining ratings or flatlined monthly income.
 - Automatically route high-churn-risk drivers (predicted churn probability > 0.65) to localized support desks for targeted surge incentives, preferential ride-matching, or fuel discount vouchers.
- **Reallocate Marketing Budget to Retention:**
 - Shift 20–25% of the driver acquisition budget toward quarterly progression bonuses for mid-grade drivers who improve their quarterly ratings.
 - Direct funds toward driver loyalty milestones (e.g., completion of 6, 12, and 24 months) rather than heavy upfront signup bonuses.
- **Operationalize XGBoost Early-Warning Pipeline:**

- Integrate the XGBoost model into monthly batch operations to generate driver-level risk scores.
- Provide regional fleet managers with a monthly dashboard highlighting high-risk drivers, replacing reactive exit management with proactive engagement.